

TechSapo システム完全ガイド

Wall-Bounce 壁打ちシステム - 使い方から技術詳細まで

TechSapo Documentation Team

2025-10-07

Contents

目次	3
第 1 部: Claude Code での使い方	3
第 2 部: システムアーキテクチャ	3
第 3 部: Wall-Bounce システムの仕組み	3
第 4 部: 詳細なシステムフロー	3
第 1 部: Claude Code での使い方	4
基本原則	4
基本的な使い方	5
タスク別の自動ルーティング	6
実際の壁打ちフロー例	7
壁打ちのルール	9
品質保証の仕組み	10
GPT-5 vs GPT-5 Codex の使い分け	11
よくある質問	12
第 2 部: システムアーキテクチャ	13
システム概要	13
高レベルアーキテクチャ	14
コンポーネント詳細	15
データフロー詳細	17
設計思想	18
技術選択理由	20
第 3 部: Wall-Bounce システムの仕組み	21
Wall-Bounce Analysis 概要	21
コアプリンシプル	21
LLM Provider 設定	22
Wall-Bounce アーキテクチャ	23

品質メトリクス	24
Wall-Bounce プロセス	25
エラーハンドリング & フォールバック	27
パフォーマンス最適化	28
設定	29
第 4 部: 詳細なシステムフロー	30
エンドツーエンドフロー	30
ルーティング決定方式	32
Wall-Bounce 実行フロー詳細	34
Early Stopping アーリーストッピング	36
LLM プロバイダー階層	37
具体的な処理例	38
詳細情報の参照先	44

目次

第 1 部: Claude Code での使い方

Wall-Bounce システムの実践的な使用方法

第 2 部: システムアーキテクチャ

全体構成とコンポーネント設計

第 3 部: Wall-Bounce システムの仕組み

複数 LLM 協調分析の技術詳細

第 4 部: 詳細なシステムフロー

エンドツーエンドの処理フロー解説

第 1 部: Claude Code での使い方

基本原則

Claude Code を使う際の絶対的な原則

物事の解決には 2 以上の LLM モデルとの壁打ち必須

すべての問い合わせで、Claude Code が司令塔となり、複数の LLM へクエリを発行して回答を統合します。

基本的な使い方

1. Claude Code に問い合わせる

あなた: 「TypeScriptでバグを修正してください」

Claude Code があなたの問い合わせを受け付けます。

2. Claude Code の内部処理 自動

Claude Code は以下を自動で実行します

ステップ 1: タスク分析

- キーワード検出: "TypeScript", "バグ修正"
- ドメイン判定: "coding"
- タスクタイプ決定: "coding"

ステップ 2: プロバイダー選択

コーディングタスク検出

↓

Tier 2 Coding特化モデルを自動選択:

- GPT-5 Codex OpenAI
- Qwen3 Coder OpenRouter
- Gemini 2.5 Flash 検証用・異なるベンダー

ステップ 3: 壁打ち実行 並列

3つのLLMへ同時にクエリ発行

↓

各LLMが独立して回答

↓

回答の類似度・品質を評価

↓

合意度 Consensus が0.6以上か

YES → 次のステップへ

NO → 追加LLMを実行 最大6ラウンド

ステップ 4: 統合

Claude Sonnet 4.5 アグリゲーター が

- 3つの回答を分析
- 最良の部分を抽出
- 一貫性のある統合回答を生成

ステップ 5: あなたへ回答

Claude Code: 「バグを修正しました。以下のコードをご確認ください...」

タスク別の自動ルーティング

コーディングタスク

あなたの問い合わせ例: - 「TypeScript で OAuth2 を実装して」 - 「このバグを修正して」 - 「コードレビューしてください」

自動選択される **LLM**: 1. GPT-5 Codex Primary 2. Qwen3 Coder Primary 3. Gemini 2.5 Flash Validation 4. Claude Sonnet 4.5 Aggregator

特徴: - 高速なコード生成 - TypeScript/JavaScript 最適化 - ミニマルな推論プロセス

分析タスク

あなたの問い合わせ例: - 「このシステムのセキュリティを分析して」 - 「アーキテクチャのトレードオフを評価して」 - 「パフォーマンス問題を調査して」

自動選択される **LLM**: 1. GPT-5 Standard Primary 2. Gemini 2.5 Pro Primary 3. Gemini 2.5 Flash Validation 4. Claude Sonnet 4.5 Aggregator

特徴: - 詳細な多段階推論 - 多角的な視点 - 構造化された分析

複雑タスク

あなたの問い合わせ例: - 「マイクロサービス移行の計画を立てて」 - 「この数学的証明を解いて」 - 「システム全体のリファクタリング戦略を提案して」

自動選択される **LLM**: 1. Gemini 2.5 Pro (Deep Think) Primary 2. GPT-5 Standard Primary 3. Claude Sonnet 4.5 Primary 4. Claude Opus 4.1 Aggregator

特徴: - 深い推論モード - 並列仮説検証 - 最も強力な統合エンジン

実際の壁打ちフロー例

例 1: コーディングタスク

あなた: 「TypeScriptでメールバリデーターを書いて」

Round 1: 並列実行

GPT-5 Codex → 「RFC5322準拠のバリデーター実装」
Qwen3 Coder → 「詳細なエラーメッセージ付き実装」
Gemini Flash → 「シンプルで高速な実装」

品質チェック

Consensus: 0.78 合格
Confidence: 0.85 高品質
→ 3つの実装が類似しており、品質も高い

統合 Claude Sonnet 4.5

3つの実装の良い部分を統合:
- GPT-5 CodexのRFC5322準拠
- Qwen3 Coderのエラーメッセージ
- Gemini Flashのシンプルさ

→ 最適な実装を生成

Claude Code → あなた

「TypeScriptのメールバリデーターを実装しました。
RFC5322に準拠し、詳細なエラーメッセージ付きです...」

例 2: 分析タスク

あなた: 「平文パスワード保存のセキュリティリスクを分析して」

Round 1: 並列実行

GPT-5 → 詳細な多段階分析
"Structuring security analysis"
"Analyzing security implications"
"Summarizing password security"

Gemini 2.5 Pro → 高品質な分析
- Core Risks
- Attack Scenarios
- Compliance Issues

Gemini 2.5 Flash → 高速な要点整理

品質チェック

Consensus: 0.72 合格
Confidence: 0.88 高品質
→ 分析内容が一致しており、信頼性が高い

統合 Claude Sonnet 4.5

3つの分析を統合：

- 最も包括的な脅威モデル
- 具体的な攻撃シナリオ
- 実践的な対策案

→ 統合されたセキュリティ分析

Claude Code → あなた

「平文パスワード保存のセキュリティリスクを分析しました。
主なリスクは…」

壁打ちのルール

最低ラウンド数: 3 回

Claude Code は必ず最低 3 回の壁打ちを実施します

1. **Round 1:** 初回 LLM 群 通常 2 つ
2. **Round 2:** 異なるベンダーの LLM で検証
3. **Round 3:** さらに異なるベンダーで検証

最大ラウンド数: 6 回

品質が基準に達しない場合、最大 6 回まで追加の LLM を実行します。

ベンダー切り替えルール

必須: 前回と異なるベンダーの LLM を選択

例:

Round 1: OpenAI GPT-5 Codex
Round 2: Google Gemini 2.5 Pro ← 異なるベンダー
Round 3: Anthropic Sonnet 4.5 ← 異なるベンダー

理由: 異なるベンダーの LLM を使うことで、バイアスを減らし、多様な視点を得る

品質保証の仕組み

コンセンサス Consensus

複数の LLM 回答がどれだけ一致しているかを示す指標。

評価方法:

各ペアの類似度を計算

↓

平均を取る

↓

0.0～1.0のスコア

判定基準: - 0.6 以上: 合格 回答が一致している - 0.6 未満: 不合格 追加ラウンド実行

信頼度 Confidence

各 LLM 回答の品質を示す指標。

評価項目: - 具体性 - 構造化 - 専門用語の適切さ - 論理的整合性

判定基準: - 0.7 以上: 高品質 - 0.5～0.7: 要検証 - 0.5 未満: 再生成

Early Stopping

品質基準を満たした時点で即座に終了し、無駄な LLM 呼び出しを回避。

条件:

Consensus $\geq$ 0.7

AND

Confidence $\geq$ 0.7

AND

実行 LLM 数 $\geq$ 必要数

↓

終了して統合ステップへ

GPT-5 vs GPT-5 Codex の使い分け

Claude Code は自動で最適なモデルを選択しますが、違いを理解しておくと有用です。

GPT-5 標準

使われるタスク: - 分析タスク - 文章生成 - 複雑な推論

特徴: - 詳細な多段階推論 - 「Structuring → Analyzing → Summarizing」の3段階思考 - 多角的な視点

推論プロセス例:

Thinking 1: "Structuring security analysis"

→ 分析の枠組みを構築中

Thinking 2: "Analyzing security implications"

→ セキュリティへの影響を分析中

Thinking 3: "Summarizing password security"

→ パスワードセキュリティをまとめ中

Output: 詳細なセキュリティ分析レポート

GPT-5 Codex

使われるタスク: - コーディング - デバッグ - コードレビュー

特徴: - ミニマルな推論 「Less is more」 - 適応推論 Adaptive Reasoning - 即座に高品質コード生成

推論プロセス例:

Thinking: "Drafting TypeScript email validator response"

→ バリデーター実装をドラフト中

Output: 即座に完成度の高いコード

自動選択の実例

あなたが「TypeScript でバグ修正」と依頼

Claude Code判定: taskType = "coding"

↓

自動選択: GPT-5 Codex

↓

特徴: ミニマル推論、即座にコード生成

あなたが「セキュリティ分析」と依頼

Claude Code判定: taskType = "analysis"

↓

自動選択: GPT-5 Standard

↓

特徴: 詳細推論、多角的分析

よくある質問

Q1: 壁打ちは毎回必須ですか

A: はい。TechSapo の絶対的原則として、すべてのクエリで最低 2 つの LLM による壁打ちを実施します。

Q2: どの LLM が選ばれるか指定できますか

A: いいえ。Claude Code がタスクタイプを自動判定し、最適な LLM を選択します。これにより、常に最高品質の回答を保證します。

Q3: 壁打ちに時間がかかりませんか

A: 並列実行により、複数 LLM を同時に呼び出すため、単一 LLM の 2 倍程度の時間で完了します。品質向上のトレードオフとして許容範囲です。

Q4: コンセンサスが低い場合はどうなりますか

A: Claude Code が自動で追加ラウンドを実行し、より多くの LLM から意見を集めます 最大 6 ラウンド。

Q5: 途中経過は見られますか

A: はい。Server-Sent Events SSE モードを使うと、各 LLM の実行状況をリアルタイムで確認できます。

第 2 部: システムアーキテクチャ

システム概要

システム特徴

- 複数 LLM 壁打ち分析: Claude、OpenAI、Gemini 等を並行実行し最適解を合議決定
- エンタープライズセキュリティ: マジックナンバー検証, 拡張子詐称対策
- 高性能最適化: 100MB/s+ 処理速度, 50ms 以内レスポンス
- 包括的テスト: 100% テスト通過による品質保証

アーキテクチャパターン

- マイクロサービス: 疎結合なコンポーネント設計
- イベント駆動: 非同期メッセージングによるスケーラビリティ
- レイヤーダアーキテクチャ: プレゼンテーション、ビジネス、データレイヤーの分離
- プラグインアーキテクチャ: LLM プロバイダーの拡張可能設計

高レベルアーキテクチャ

システム全体構成

外部サービス層:

- Google Drive API
- OpenAI API
- Anthropic Claude
- Google Gemini

フロントエンド層:

- Web UI
- CLI Interface
- API Gateway

アプリケーション層:

- Authentication Service
- Wall Bounce Orchestrator
- File Parser Engine
- RAG Search Engine

データ層:

- Vector Store
- Redis Cache
- PostgreSQL
- File Storage

監視・運用層:

- Prometheus
- Grafana
- AlertManager

データフロー

フロントエンド → **API Gateway**

ユーザーからのリクエストを受け付け、認証・ルーティング

API Gateway → **Wall Bounce Orchestrator**

分析クエリを複数LLMへ分散実行

Wall Bounce → **LLM Providers**

OpenAI、Anthropic、Geminiへ並列リクエスト

LLM Providers → **Consensus Engine**

各LLMからの回答を収集・評価

Consensus Engine → **Aggregator**

統合された最終回答を生成

コンポーネント詳細

1. Wall Bounce Orchestrator

責任: 複数 LLM による協調分析の調整

主要機能: - クエリ受信・前処理 - 複数 LLM へ並列リクエスト送信 - レスpons収集・品質評価 - 合意アルゴリズムによる最終回答生成 - 信頼度スコア算出・メタデータ付与

設定項目:

- models: 使用する LLM プロバイダーのリスト
- consensusStrategy: 'majority' | 'weighted' | 'unanimous'
- minConfidence: 最低信頼度 0.7
- parallelExecution: 並列実行の有効化

結果フォーマット:

- answer: 統合された回答テキスト
- confidence: 信頼度スコア 0.0-1.0
- consensus: 合意情報
- metadata: 分析メタデータ

2. Authentication Service

責任: ユーザー認証・アクセス制御

機能: - OAuth2.0 / JWT 認証 - セッション管理 - ロールベースアクセス制御 RBAC - API キー管理

3. RAG Search Engine

責任: Google Drive 統合による高精度文書検索

主要コンポーネント:

Drive Synchronizer

Google Drive API との同期

- ファイル一覧取得
- 変更検出
- 自動同期

Vector Indexer

OpenAI Vector Store へのインデックス作成

- テキスト抽出
- ベクトル化
- インデックス登録

Hybrid Retriever

密なベクトル + スパースキーワード検索

- ベクトル類似度検索
- キーワードマッチング
- 結果統合

Context Ranker

検索結果の関連度順位付け

- スコアリング
- ランキング
- コンテキスト抽出

4. Monitoring & Metrics

責任: システム性能監視・品質保証

メトリクス分類:

壁打ち分析メトリクス

- success_rate: 成功率
- confidence_score: 信頼度分布
- response_time: レスポンス時間

システムメトリクス

- http_requests_total: リクエスト総数
- http_request_duration: リクエスト所要時間
- memory_usage: メモリ使用量

データフロー詳細

1. 壁打ち分析フロー

ステップ1: ユーザーがAnalysis Queryを送信
↓
ステップ2: API Gatewayがリクエストを受信
↓
ステップ3: Wall Bounce OrchestratorがProcess Request
↓
ステップ4: 並列LLM呼び出し
- OpenAI GPT-4へSend Query
- Claude-3へSend Query
- Gemini ProへSend Query
↓
ステップ5: レスポンス収集
- LLM1からResponse A
- LLM2からResponse B
- LLM3からResponse C
↓
ステップ6: Consensus Engineでコンセンサス評価
↓
ステップ7: Final Answerを生成
↓
ステップ8: API GatewayがUnified Responseを返却
↓
ステップ9: ユーザーが結果を受け取る

2. RAG 検索フロー

ステップ1: ユーザーがSearch Queryを送信
↓
ステップ2: API Gatewayがリクエストを受信
↓
ステップ3: RAG EngineがProcess Search
↓
ステップ4: Google DriveからSync Latest Files
↓
ステップ5: Vector StoreでVector Search
↓
ステップ6: Relevant Documentsを取得
↓
ステップ7: LLM ProviderでGenerate Answer
↓
ステップ8: Contextual Responseを生成
↓
ステップ9: Search Resultsを返却
↓
ステップ10: ユーザーがAnswer + Sourcesを受け取る

設計思想

1. 信頼性優先設計

冗長性

複数LLMによる相互検証

- 単一障害点の排除
- フェイルオーバー機能

フェイルセーフ

単一障害点の排除

- 自動リトライ
- 代替プロバイダーへの切り替え

グレースフルデグラデーション

部分機能停止時の継続動作

- 最低限の機能維持
- ユーザーへの透明な通知

2. セキュリティファースト

多層防御

- ファイル検証: マジックナンバー、拡張子チェック
- 実行時保護: サンドボックス実行
- 監査ログ: 全操作の記録

ゼロトラスト

全ての入力を検証・検疫

- 信頼しない
- 常に検証
- 最小権限

最小権限原則

必要最低限のアクセス権限

- ロールベースアクセス制御
- 時限付きトークン

3. パフォーマンス最適化

非同期処理

I/O待機時間の最小化

- Promise.all()による並列実行
- Worker Poolによるタスク分散

キャッシング戦略

多層キャッシュによる高速化

- L1: メモリキャッシュ
- L2: Redisキャッシュ
- L3: CDNキャッシュ

リソース効率

メモリ・CPU使用量の最適化

- ストリーミング処理
- バッチ処理
- ガベージコレクション最適化

4. 拡張性・保守性

プラグインアーキテクチャ

新機能の容易な追加

- プロバイダーインターフェース
- ファクトリーパターン

設定駆動

外部設定による動作変更

- 環境変数
- 設定ファイル
- 動的リロード

テスト駆動開発

100%品質保証

- ユニットテスト
- 統合テスト
- E2Eテスト

技術選択理由

プログラミング言語・ランタイム

TypeScript/Node.js - 型安全性 - 高いエコシステム - 非同期処理性能 - LLM API との統合容易性 - JSON 処理性能 - 豊富なライブラリ

データベース・ストレージ

PostgreSQL - ACID 準拠 - 複雑クエリ - JSON 型サポート

Redis - 高速キャッシング - セッション管理 - Pub/Sub 機能

Google Cloud Storage - スケーラブルファイルストレージ - 高可用性 - コスト効率

OpenAI Vector Store - 高精度ベクトル検索 - マネージドサービス - API シンプル

LLM プロバイダー

OpenAI GPT-5/Codex - 高い推論能力 - 広範囲な知識 - コーディング特化版

Anthropic Claude - 長文処理 - 倫理的配慮 - 詳細な分析

Google Gemini - マルチモーダル - 高速処理 - コスト効率

選択理由: 各 LLM の特性を活かした補完的な構成

インフラ・デプロイメント

Docker/Kubernetes - コンテナ化 - オーケストレーション - スケーラビリティ

Google Cloud Platform - マネージドサービス - スケーラビリティ - 高可用性

Prometheus/Grafana - メトリクス監視 - 可視化 - アラート機能

選択理由: クラウドネイティブ・高可用性・運用効率

第 3 部: Wall-Bounce システムの仕組み

Wall-Bounce Analysis 概要

Wall-Bounce 分析システムは、複数の LLM プロバイダーを協調させて高品質な回答を生成する中核機能です。

コアプリンシプル

必須ルール

最低 2 プロバイダー

すべての分析で最低2つのLLMプロバイダーを使用
理由: 単一LLMのバイアスを排除し、多様な視点を確保

コンセンサス必須

プロバイダー間での合意形成が必要
閾値: コンセンサス ≥ 0.6

品質閾値

信頼度 ≥ 0.7
コンセンサス ≥ 0.6

日本語応答

基本的に日本語での回答生成
英語クエリには英語で応答

LLM Provider 設定

OpenAI

モデル: GPT-5 専用 GPT-4/GPT-4o 使用禁止

設定:

```
model: 'gpt-5'  
temperature: 0.7  
max_tokens: 2000
```

用途: - 一般分析タスク GPT-5 Standard - コーディングタスク GPT-5 Codex

Anthropic

使用方法: SDK 使用のみ API_KEY 禁止

理由: MAX x5 Plan のコスト回避

設定:

SDK経由でのみ使用
API_KEYは使用しない
SDK専用キー使用

用途: - 長文処理 - 詳細な分析 - 倫理的配慮が必要なタスク

Google Gemini

モデル: Gemini 2.5 シリーズ

バリエーション:

- gemini-2.5-flash: 高速・低コスト
- gemini-2.5-pro: 高品質分析
- gemini-2.5-pro (Deep Think): 深い推論

設定:

```
temperature: 0.8  
maxTokens: 1500
```

用途: - 高速分析 - マルチモーダル処理 - コスト効率重視タスク

Wall-Bounce アーキテクチャ

分析フロー

入力： ユーザークエリ
↓
ステップ1: Wall-Bounce Orchestratorで処理
↓
ステップ2: 並列LLM呼び出し
 - GPT-5 Analysis
 - Gemini Analysis
 - Claude Analysis
↓
ステップ3: Consensus Engineで評価
 - Quality Scoring
 - 合意度計算
↓
ステップ4: Response Integration
↓
出力： 統合された回答

タスクタイプとプロバイダー選択

Basic Task 基本タスク

プロバイダー数： 2
使用モデル： GPT-5 + Gemini 2.5 Flash
信頼度閾値： 0.7
特徴： コスト優先、低コストモデル選択

Premium Task プレミアムタスク

プロバイダー数： 3
使用モデル： GPT-5 + Gemini 2.5 Pro + Claude (SDK)
信頼度閾値： 0.8
特徴： バランス重視、品質とコストのバランス

Critical Task クリティカルタスク

プロバイダー数： 3-4
使用モデル： 全プロバイダー + OpenRouter
信頼度閾値： 0.9
特徴： 品質優先、最高品質の分析

品質メトリクス

Confidence 信頼度

定義: 回答の信頼度を示す指標 0.0-1.0

評価方法: 各プロバイダーが以下を総合的に評価

- 回答の確実性
- 情報源の信頼性
- 推論の論理的ー貫性

閾値:

- 0.9以上: 非常に高い信頼度
- 0.7-0.9: 高い信頼度 合格
- 0.5-0.7: 中程度 要検証
- 0.5未満: 低い信頼度 再生成

Consensus 合意度

定義: プロバイダー間の回答類似度 0.0-1.0

評価方法: 各プロバイダーペアの類似度を計算し平均

例: 3プロバイダー中2つが合意
→ $\text{consensus} = 2/3 = 0.67$

閾値:

- 0.7以上: 強い合意 理想的
- 0.6-0.7: 十分な合意 合格
- 0.6未満: 不十分 追加ラウンド

Coherence ー貫性

定義: 回答内の論理的ー貫性 0.0-1.0

評価項目:

- 主張と根拠の整合性
- 矛盾の有無
- 結論の妥当性

Completeness 完全性

定義: 回答の網羅性 0.0-1.0

評価項目:

- 質問への完全な回答
- 必要な情報の包含
- フォローアップの提供

Wall-Bounce プロセス

Step 1: Query Analysis クエリ分析

分析項目:

complexity: 'basic' | 'premium' | 'critical'

- タスクの複雑度を判定

domain: 'technical' | 'business' | 'creative'

- ドメインを分類

language: 'japanese' | 'english'

- 言語を判定

urgency: 'low' | 'medium' | 'high'

- 緊急度を評価

Step 2: Provider Selection プロバイダー選択

選択ロジック:

Primary: ['gpt-5', 'gemini-2.5-flash']

- 初回実行用の主要プロバイダー

Secondary: ['claude-sonnet', 'gemini-2.5-pro']

- 追加検証用プロバイダー

Fallback: ['openrouter-models']

- フェイルオーバー用

Step 3: Parallel Execution 並列実行

実行方法:

Promise.allで並列実行:

- analyzeWithGPT5(query, context)
- analyzeWithGemini(query, context)
- analyzeWithClaude(query, context)

メリット:

- レスポンス時間の短縮
- 各LLMの独立性確保
- 効率的なリソース利用

Step 4: Quality Assessment 品質評価

評価ステップ:

1. Individual Scores

各プロバイダーの個別スコア算出

2. Cross Validation

プロバイダー間の相互検証

3. Consensus Level

合意度の測定

4. Final Confidence

最終信頼度の計算

Step 5: Response Integration 回答統合

統合処理:

Content:

- 合意された回答を重み付きで統合

Confidence:

- 最終信頼度スコアを算出

Reasoning:

- 決定プロセスの説明を生成

Metadata:

- 使用プロバイダー
- 処理時間
- コスト内訳

エラーハンドリング & フォールバック

Provider Failures プロバイダー障害

Single Failure 単一障害

動作: 残りのプロバイダーで継続

理由: 冗長性により品質維持

Multiple Failures 複数障害

動作: プレミアムプロバイダーへエスカレーション

理由: 品質確保のため上位層を使用

Complete Failure 完全障害

動作: コンテキスト付きエラーを返却

理由: ユーザーへの透明な通知

Quality Thresholds 品質閾値

Consensus < 0.6 の場合

処理: 追加プロバイダーでの再分析

理由: 合意度が不十分

Confidence < 0.7 の場合

処理: 自動エスカレーション

理由: 信頼度が基準未滿

パフォーマンス最適化

Caching Strategy キャッシング戦略

Query Caching

- 類似クエリのキャッシュ利用
- TTL: 1時間
- キャッシュキー: クエリのハッシュ値

Provider Caching

- プロバイダー応答の一時保存
- TTL: 30分
- キャッシュキー: プロバイダー名 + クエリハッシュ

Consensus Caching

- 合意結果の再利用
- TTL: 1時間
- キャッシュキー: プロバイダーセット + クエリハッシュ

Cost Optimization コスト最適化

Smart Routing

コスト効率を考慮したプロバイダー選択

- 低コストモデル優先
- 必要に応じて高コストモデル使用

Batch Processing

複数クエリのまとめ処理

- バッチAPIの活用
- レートリミット対策

Budget Management

リアルタイムコスト監視

- 月次予算: \$70
- アラート閾値: 80% \$56
- 自動制限: 95% \$66.5

設定

環境変数

Wall-Bounce Configuration

WALL_BOUNCE_MIN_PROVIDERS=2

最低プロバイダー数

WALL_BOUNCE_MAX_PROVIDERS=4

最大プロバイダー数

WALL_BOUNCE_CONFIDENCE_THRESHOLD=0.7

信頼度閾値

WALL_BOUNCE_CONSENSUS_THRESHOLD=0.6

合意度閾値

Provider-Specific Settings

OPENAI_MODEL=gpt-5

GPT-5 only GPT-4/4o禁止

ANTHROPIC_USE_SDK=true

SDK使用 API_KEY禁止

ANTHROPIC_API_KEY_DISABLED=true

API_KEY明示的無効化

GEMINI_MODEL=gemini-2.5-flash

Geminiデフォルトモデル

Task Configuration タスク設定

Basic Task

minProviders: 2

maxProviders: 2

confidenceThreshold: 0.7

budgetTier: 'standard'

Premium Task

minProviders: 3

maxProviders: 3

confidenceThreshold: 0.8

budgetTier: 'premium'

Critical Task

minProviders: 3

maxProviders: 4

confidenceThreshold: 0.9

budgetTier: 'unlimited'

第 4 部: 詳細なシステムフロー

エンドツーエンドフロー

全体フロー概要

ステップ1: ユーザークエリ

「TypeScriptでバグ修正してください」

↓

ステップ2: タスクタイプ判定

- キーワード検出: "TypeScript", "バグ修正"
 - ドメイン判定: domain = "coding"
 - 複雑度評価: 通常タスク
- 判定結果: taskType = "coding", complexity = "normal"

↓

ステップ3: LLMプロバイダー選択

IF taskType == "coding":
→ Tier 2 Coding専用モデルを選択
→ providers = ["gpt-5-codex", "qwen3-coder"]

ELSE IF taskType == "analysis":
→ Tier 1/2 General モデルを選択
→ providers = ["gpt-5", "gemini-2.5-pro"]

選択結果: ["gpt-5-codex", "qwen3-coder"]

↓

ステップ4: Wall-Bounce 並列実行

3つのLLMへ同時にクエリ発行:

- GPT-5 Codex (Tier 2)
- Qwen3 Coder (Tier 2)
- Gemini 2.5 Pro (Tier 1, 検証用)

各LLMが独立して回答:

- Response 1 + メタデータ
- Response 2 + メタデータ
- Response 3 + メタデータ

Quorum Judge コンセンサス判定 :

- 類似度計算
- 品質スコア算出
- k=2 合意チェック

Consensus $\geq$ 0.6?

- YES → 次のステップへ
- NO → 追加LLM実行

↓

ステップ5: アグリゲーター統合

IF complexity < 2:
→ aggregator = "sonnet-4.5" (Tier 3)

ELSE IF complexity ≥ 2 :
→ aggregator = "opus-4.1" (Tier 4)

Claude Sonnet 4.5 (Default Aggregator):

Input:

- 全プロバイダー回答
- コンセンサススコア
- 元のクエリ

Processing:

- 矛盾の解消
- 重複の統合
- 最終推奨の生成

Output:

- 統合された最終回答
- メタデータ
- 品質スコア

↓

ステップ6: 最終レスポンス

```
{  
  "answer": "統合された回答テキスト",  
  "confidence": 0.85,  
  "consensus": 0.78,  
  "providersUsed": ["gpt-5-codex", "qwen3-coder", "gemini"],  
  "aggregator": "sonnet-4.5"  
}
```

ルーティング決定方式

Phase 1: タスクタイプ判定

キーワード検出ロジック:

Coding判定キーワード:

- "TypeScript", "バグ"
 - "コード", "実装"
 - "debug", "fix"
- taskType = "coding"

Analysis判定キーワード:

- "分析", "調査"
 - "セキュリティ", "評価"
 - "analyze", "review"
- taskType = "analysis"

ドメイン明示:

options.domain === "coding" → Coding
options.domain === "analysis" → Analysis

デフォルト:

不明な場合 → "basic" (汎用)

Phase 2: LLM プロバイダー選択

Coding タスク:

Primary (Tier 2 - Coding特化):

- ✓ gpt-5-codex
- ✓ qwen3-coder

Validation (Tier 1 - 異なるベンダー):

- ✓ gemini-2.5-flash

Analysis タスク:

Primary (Tier 1/2 - General):

- ✓ gpt-5 (standard)
- ✓ gemini-2.5-pro

Validation (Tier 1 - 高速):

- ✓ gemini-2.5-flash

Basic タスク:

Cost-Efficient (Tier 1):

- ✓ gemini-2.5-flash
- ✓ gpt-5 (if needed)

Critical タスク:

All Available (Tier 1-4):

- ✓ gemini-2.5-pro (Deep Think)
- ✓ gpt-5 / gpt-5-codex
- ✓ sonnet-4.5
- ✓ opus-4.1

Phase 3: アグリゲーター選択

複雑度スコア計算:

```
score = 0
```

```
IF キーワード含む: "architect", "design"  
  → score += 1
```

```
IF 日本語キーワード含む: "複雑な", "高度な", "詳細な"  
  → score += 1
```

```
IF プロンプト長 > 500文字  
  → score += 1
```

```
IF 疑問符 ≥ 3個  
  → score += 1
```

選択ロジック:

```
IF score < 2:  
  aggregator = "sonnet-4.5" ← Tier 3  
  (標準アグリゲーター)
```

```
ELSE IF score ≥ 2:  
  aggregator = "opus-4.1" ← Tier 4  
  (複雑タスク専用)
```

Wall-Bounce 実行フロー詳細

Step 1: プロバイダー並列実行

プロンプト生成 Provider Guidance 付与 :

GPT-5 Codex用 プロンプト:

「実装方針や設定変更の手順を具体的に示してください。必要に応じてコードスニペットやコマンド例を提示してください。」

+ ユーザークエリ

Qwen3 Coder用 プロンプト:

「TypeScriptのベストプラクティスに沿って具体的なコード例を示してください。差分形式や検証ステップがある場合は明記し、潜在的な副作用も指摘してください。」

+ ユーザークエリ

並列実行:

Provider 1: GPT-5 Codex → [実行中...]

Provider 2: Qwen3 Coder → [実行中...]

Provider 3: Gemini 2.5 Pro → [実行中...]

↓

Response 1: "まず..." (confidence: 0.85)

Response 2: "以下の..." (confidence: 0.82)

Response 3: "TypeScriptの修正..." (confidence: 0.78)

Step 2: Quorum Judge コンセンサス判定

設定:

k = 2 (必要合意数)

scoreThreshold = 0.82

abstainBelow = 0.5

処理:

1. 類似度計算

Response1 vs Response2: 0.75

Response1 vs Response3: 0.68

Response2 vs Response3: 0.80

2. 合意グループ検出

Group A: [Response1, Response2]

(類似度 0.75, size=2)

3. Quorum達成チェック

Group A size (2) ≥ k (2) ✓

→ Quorum達成

結果:

```
quorumAchieved: true
winningResponses: [Response1, Response2]
consensusScore: 0.75
metadata: {
  totalProviders: 3,
  agreedProviders: 2
}
```

Step 3: アグリゲーター統合

Input:

- 元のクエリ
- Winning Responses (2件)
- Consensus Score: 0.75
- Provider Metadata

Aggregation Prompt:

「以下の各LLM回答を統合し、
矛盾があれば整合させてください。
重複内容は統合し、最終的な
推奨行動・留意点・フォロー
アップを明確にしてください。」

Response 1 (GPT-5 Codex):
[内容...]

Response 2 (Qwen3 Coder):
[内容...]

Processing:

1. 矛盾点の検出・解消
2. 重複内容の統合
3. 最終推奨の生成
4. 品質スコアの付与

Output:

「TypeScriptのバグ修正について

【修正手順】

1. [統合された手順]
2. ...

【コード例】

[統合されたコード]

【注意点】

- [重要な留意事項]

【フォローアップ】

- [推奨される次のステップ]

Early Stopping アーリーストッピング

フロー

設定:

minProviders = 2
maxProviders = 5
k = 2 (必要合意数)

実行フロー:

Round 1: 2 providers実行

↓

Quorum達成? (size ≥ k)

└ YES → 終了 (高速完了) ✓
└ NO → Round 2へ

Round 2: +1 provider実行 (total 3)

↓

Quorum達成?

└ YES → 終了 ✓
└ NO → Round 3へ

...

Round N: maxProvidersに達するまで繰り返し

メリット:

- ✓ コスト削減 (不要なLLM呼び出し回避)
- ✓ レスポンス速度向上
- ✓ 品質は維持 (合意閾値で保証)

LLM プロバイダー階層

Tier アーキテクチャ

Tier 0: ドキュメント参照 非 LLM

- Context7: ライブラリドキュメント検索
- Stash: コードベース参照

用途: API仕様、ライブラリ使用方法の参照

Tier 1: 高速・コスト効率 Primary Analysis

- Gemini 2.5 Pro (Deep Think): 深い推論
- Gemini 2.5 Pro: 高品質分析
- Gemini 2.5 Flash: 高速分析

特徴: 低コスト、高速、マルチリンガル

用途: 初期分析、情報整理、要約

Tier 2: 専門特化 Specialized Tasks

Coding専用:

- GPT-5 Codex: OpenAI Codex CLI (適応推論)
- Qwen3 Coder: 480B MoE (35B active, Agentic)

General:

- GPT-5 (標準): 詳細推論・分析

特徴: タスク特化、高品質、ベストプラクティス内蔵

用途: コーディング、デバッグ、専門分析

Tier 3: 標準アグリゲーター Response Integration

- Claude Sonnet 4.5: 標準統合・高速推論

特徴: バランス型、複数回答統合、矛盾解消

用途: 通常の複雑度タスクの最終統合

Tier 4: 複雑タスク専用 Complex Aggregation

- Claude Opus 4.1: 最高品質統合

特徴: 最高品質、複雑な論理統合、アーキテクチャ設計

用途: 高複雑度タスク、セキュリティ監査、設計判断

具体的な処理例

例 1: コーディングタスク

入力:

ユーザークエリ: "TypeScriptでEmailバリデーション関数を実装して"
domain: "coding"

処理フロー:

Step 1: タスクタイプ判定

キーワード: "TypeScript", "実装" → Coding
ドメイン: "coding" → Coding確定
→ taskType = "coding"

Step 2: プロバイダー選択

Coding タスク検出
→ Tier 2 Coding モデル選択

Selected:

1. GPT-5 Codex (OpenAI)
2. Qwen3 Coder (OpenRouter)
3. Gemini 2.5 Flash (検証用)

Step 3: 並列実行

GPT-5 Codex → Response 1:
export function isValidEmail(email: string): boolean { ... }
confidence: 0.88

Qwen3 Coder → Response 2:
function validateEmail(email: string): boolean { ... }
confidence: 0.85

Gemini Pro → Response 3:
const validateEmail = (email: string) => { ... }
confidence: 0.80

Step 4: Quorum Judge

類似度計算:

- Response1 vs Response2: 0.82 ✓
- Response1 vs Response3: 0.75
- Response2 vs Response3: 0.78

合意グループ:

- Group A: [Response1, Response2] (size=2, similarity=0.82)

Quorum達成: YES ($2 \geq k=2$)
Consensus Score: 0.82

Step 5: アグリゲーター

複雑度スコア: 0 (低)
→ Aggregator: Claude Sonnet 4.5

統合処理:
- Response1 (GPT-5 Codex)
- Response2 (Qwen3 Coder)

↓

最終回答:
TypeScriptでのEmailバリデーション実装:

```
```typescript
export function isValidEmail(email: string): boolean {
 if (!email) return false;
 const normalized = email.trim();
 if (normalized.length > 254) return false;
 return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalized);
}
```

【実装のポイント】 1. 型安全性: TypeScript の型定義活用 2. エッジケース: 空文字・長さ制限 3. RFC5322 準拠の正規表現パターン

【テスト例】 [テストコード]

メタデータ: - confidence: 0.88 - consensus: 0.82 - providersUsed: [ gpt-5-codex , qwen3-coder ] - aggregator: sonnet-4.5

\newpage

### 例2: 複雑な分析タスク

\*\*入力\*\*:

ユーザークエリ: マイクロサービスアーキテクチャへの移行における セキュリティリスクを詳細に分析し、対策を提案して  
domain: analysis

\*\*処理フロー\*\*:

\*\*Step 1: タスクタイプ判定\*\*

キーワード: 分析, セキュリティ → Analysis ドメイン: analysis → Analysis 確定 → taskType = analysis

\*\*Step 2: 複雑度評価\*\*

score = 0

キーワード含む: - architect → +1 - security → +1

日本語キーワード含む: - 詳細 → +1

プロンプト長: 72 文字 (< 500) 疑問符: 0 個

総スコア: 3 (高複雑度)

→ complexity = high → Aggregator: Opus 4.1 (Tier 4)

**\*\*Step 3: プロバイダー選択\*\***

Analysis + High Complexity → Tier 1 + Tier 2 + 異なるベンダー

Selected: 1. Gemini 2.5 Pro (Deep Think) 2. GPT-5 (標準) 3. Claude Sonnet 4.5 (分析用)

**\*\*Step 4: 並列実行\*\***

3 プロバイダーが並列で分析実行

各プロバイダーが以下を提供: - セキュリティリスクの分析 - 対策の提案 - 実装の推奨事項

**\*\*Step 5: Quorum Judge\*\***

3 つの回答の類似度評価 - 共通リスク項目の抽出 - 対策の整合性確認 - Consensus Score: 0.78

**\*\*Step 6: アグリゲーター (Opus 4.1)\*\***

高複雑度タスク → Opus 4.1 使用

統合処理: - 各回答からリスクを網羅的に統合 - 矛盾する推奨事項を調整 - 優先順位付き対策リストを生成

↓

最終回答: マイクロサービス移行のセキュリティ分析

【主要リスク】1. サービス間通信の脆弱性 - API Gateway 認証の欠如 - 暗号化されていない内部通信

2. 分散認証・認可の課題

- トークン管理の複雑化
- セッション同期の問題

[ ]

【優先対策】Priority 1: (即時実施) - mTLS 導入によるサービス間通信保護 - OAuth2.0 + JWT による統一認証基盤

Priority 2: (1 ヶ月以内) - API ゲートウェイでのレート制限 - サービスメッシュ導入検討

[ ]

【実装ロードマップ】[詳細なステップ ]

メタデータ: - confidence: 0.92 - consensus: 0.78 - providersUsed: [ gemini-2.5-pro , gpt-5 , sonnet-4.5 ] - aggregator: opus-4.1

\newpage

## 品質保証の仕組み

### 品質メトリクス

**\*\*Confidence 信頼度 \*\*:** 0.0 ~ 1.0

各プロバイダーが自己評価で算出:



評価項目: - 回答の確実性 - 情報源の信頼性 - 推論の論理的一貫性

算出方法: 複数の要素を総合的に評価して決定

**\*\*Consensus 合意度 \*\*:** 0.0 ~ 1.0

プロバイダー間の回答類似度:

評価方法: 各合意グループのサイズを総プロバイダー数で割る

例: 3 プロバイダー中 2 つが合意 →  $\text{consensus} = 2/3 = 0.67$

**\*\*Coherence 一貫性 \*\*:** 0.0 ~ 1.0

回答内の論理的一貫性:

評価項目: - 主張と根拠の整合性 - 矛盾の有無 - 結論の妥当性

**\*\*Completeness 完全性 \*\*:** 0.0 ~ 1.0

回答の網羅性:

評価項目: - 質問への完全な回答 - 必要な情報の包含 - フォローアップの提供

**\*\*最終品質スコア\*\*:**

複数のメトリクスを総合的に評価して算出 主要要素: Confidence と Consensus

\newpage

### 品質判定フローチャート

**\*\*ステップ1: Consensus チェック\*\***

IF  $\text{consensus} < 0.6$ : → 追加プロバイダー実行 → 最大  $\text{maxProviders}$  まで繰り返し

ELSE IF  $\text{consensus} \geq 0.6$ : → ステップ 2 へ

**\*\*ステップ2: Confidence チェック\*\***

IF  $\text{avg\_confidence} < 0.7$ : → 警告付きで応答 → メタデータに低信頼度フラグ

ELSE IF  $\text{avg\_confidence} \geq 0.7$ : → ステップ 3 へ

**\*\*ステップ3: アグリゲーション\*\***

- 合意回答を統合
- 最終品質スコア算出
- メタデータ付与

#### **\*\*ステップ4: 最終チェック\*\***

IF final\_quality\_score < 0.65: → 品質警告を含めて応答 → ユーザーに再試行を提案

ELSE: → 高品質応答として返却

\newpage

### **### 品質保証のベストプラクティス**

#### **\*\*1. 最低プロバイダー数の保証\*\***

minProviders 2 (常に) Critical タスクは 3-4 プロバイダー

#### **\*\*2. ベンダー多様性\*\***

異なるベンダーの LLM を混在 OpenAI → Google → Anthropic ベンダー固有バイアスを相殺

#### **\*\*3. Tier ベースの役割分担\*\***

Tier 1/2: 専門分析 Tier 3/4: 統合・品質向上

#### **\*\*4. Early Stopping による効率化\*\***

Quorum 達成時に即座に終了 コスト削減とレスポンス速度向上

#### **\*\*5. メタデータの完全性\*\***

使用プロバイダーの記録 スコア詳細の保存 トレーサビリティの確保

\newpage

### **## まとめ**

### **### システムの強み**

#### **\*\*1. 高品質保証\*\***

複数 LLM のコンセンサスにより、単一 LLM より高品質な回答を生成

#### **\*\*2. 自動最適化\*\***

タスクタイプに応じた最適な LLM 選択で、品質とコストを両立

#### **\*\*3. スケーラビリティ\*\***

Tier 構造により、新しい LLM の追加が容易

#### \*\*4. トレーサビリティ\*\*

すべての判断プロセスを メタデータとして記録

#### ### 期待される効果

##### \*\*回答品質\*\*

単一 LLM 比で 15-20% 向上

##### \*\*一貫性\*\*

コンセンサスメカニズムにより 安定した品質

##### \*\*コスト効率\*\*

Early Stopping で 30-40% のコスト削減

##### \*\*レスポンス速度\*\*

並列実行で高速化 単一 LLM 比 60-70% の時間 “

## 詳細情報の参照先

より詳しい情報が必要な場合は、以下のドキュメントを参照してください

### システム詳細

- [/ai/prj/techdev/docs/SYSTEM\\_FLOW\\_EXPLANATION.md](#) - 詳細なシステムフロー
- [/ai/prj/techdev/docs/WALL\\_BOUNCE\\_SYSTEM.md](#) - Wall-Bounce の技術詳細

### アーキテクチャ

- [/ai/prj/techdev/docs/ARCHITECTURE\\_OVERVIEW.md](#) - システムアーキテクチャ
- [/ai/prj/techdev/docs/LLM\\_PROVIDERS\\_GUIDE.md](#) - LLM プロバイダーガイド

## GPT-5/Codex

- [/ai/prj/techdev/docs/GPT5\\_CODEX\\_FINAL\\_SUMMARY.md](#) - 検証結果サマリー
- [/ai/prj/techdev/docs/GPT5\\_VS\\_GPT5\\_CODEX\\_実証結果.md](#) - 使い分け実証

### トラブルシューティング

- [/ai/prj/techdev/docs/TROUBLESHOOTING.md](#) - 問題解決ガイド

### 開発者向け

- [/ai/prj/techdev/CLAUDE.md](#) - Claude Code 開発ガイド
- [/ai/prj/techdev/docs/API\\_REFERENCE.md](#) - API 仕様

---

作成: TechSapo Documentation Team 更新: 2025-10-07 バージョン: 2.0